

Completed 700 requests
 Completed 800 requests
 Completed 900 requests
 Completed 1000 requests
 Finished 1000 requests

Server Software: Apache/2.4.33
 Server Hostname: X.X.X.X
 Server Port: 80

Document Path: /cgi-perl/printenv
 Document Length: 756 bytes

Concurrency Level: 100
 Time taken for tests: 2.758 seconds
 Complete requests: 1000
 Failed requests: 0
 Total transferred: 962000 bytes
 HTML transferred: 756000 bytes
 Requests per second: 362.54 [#/sec] (mean)
 Time per request: 275.831 [ms] (mean)
 Time per request: 2.758 [ms] (mean, across all concurrent requests)
 Transfer rate: 340.59 [Kbytes/sec] received

Connection Times (ms)					
	min	mean	[+/-sd]	median	max
Connect:	0	10	17.9	1	63
Processing:	19	254	41.0	258	425
Waiting:	14	247	41.0	254	424
Total:	30	264	34.1	262	429

Percentage of the requests served within a certain time (ms)	
50%	262
66%	272
75%	277
80%	281
90%	296
95%	314
98%	322
99%	353
100%	429 (longest request)

What we have learnt so far is that:

1. We can load Perl modules for using many of Apache's features using *mod_perl*, which delivers better Web performance and threading.
2. Even without loading any Perl module:
 - a) When it comes to the time taken for tests, *mod_perl* is six times lesser than *mod_cgi*.
 - b) With respect to the transfer rate and length of request time period, *mod_perl* is five times greater than *mod_cgi*.
3. *mod_cgi* uses less RAM compared to *mod_perl*. Bugzilla.com claims that for *mod_perl*, "The more RAM you

can get, the better. *mod_perl* is basically trading RAM for speed. At least 2GB total system RAM is recommended for running Bugzilla under *mod_perl*."

4. As mentioned in the last article, *mod_perl* is only supported till Apache 2.2, though it works in Apache 2.4. *mod_cgi* and *mod_env* are supported by default in all Apache versions.

There are trade-offs and compromises to be made in the case of both the modules. *mod_perl* is more popular because of its ability to load Perl modules as part of the Apache startup by using the following:

- PerlModule variable, or
- PerlRequire <script_path>

These are kept in the memory/cache for any CGI Perl program. Big companies like Amazon.com (Perl's Mason template) and Bugzilla.com use *mod_perl*.

Experiments with the `env` command

If we were to comment out in the *printenv* script, as follows:

```
#foreach $var (sort(keys(%ENV))) {
# print "$var=".$ENV{$var}."\n";
#}
```

...and type the following command:

```
print `env`;
```

...how different would the output look? Try it out (<http://IPAddress/cgi-bin/printenv>, <http://IPAddress/cgi-perl/printenv>).

Adding alignment and headers

This is just to create a better look and feel. For this, let's use cases 2 and 4 from our earlier setup (<http://IPAddress/cgi-perl/printenv-html> and <http://IPAddress/cgi-bin/printenv-html>).

After we have opened one of these, right click on the page. Then, select 'View source' in IE, and 'View page source' in Chrome/Firefox.

We will see that the page looks cluttered with no proper spacing.

Edit the script and add `\n` to the right of `<br>`. The output will look like what follows:

```
#!/usr/local/bin/perl

print "Content-type: text/html\n\n";
foreach $var (sort(keys(%ENV))) {
  print "$var=".$ENV{$var}."<br>\n";
}
```

Now, view the source again, as before. You should be able to see each variable and its value, on each line, rather than all in one line. This will help in troubleshooting Web pages later on, and it also tidies up your page.

You will notice there are no headers (e.g., `<html>`, `<body>`, etc), and also that the tab of the browser does not have a title;